REQUIREMENT ANALYSIS

1. Introduction

1.1. Purpose of the system

This document defines the specifications and expectations of the development of a videogame, tentatively called "IFP-Tanks". The game is created for the advanced pratical "Agile development of computer games" of Paul Brinkmann, Gero Brunke and Nils Schlaback in University Heidelberg.

1.2. Scope of the system

The scope of the video game encompasses the design and development of an top-down game in which the player is controlling a tank needing to eliminate all enemy tanks. The game is programmed using the game engine Godot with the language C#.

Key elements of the game include:

- Multiple individually designed levels grouped in worlds
- Multiple gimicks for each world like switches and gates, teleporter and destroyable walls
- A variety of enemy tanks with different abilities
- Upgrades for player controlled tank
- Pixelart graphics

1.3. Objectives and success criteria of the project

The Objectives and Success Criteria of the project aim to ensure that students gain first practical experiences producing a well-designed and functional video game. The success is measured by following criteria:

- Fully functional video game which meets the defined project goals and requirements.
- The game runs without major technical difficulties.
- The students gain first experience using agile development strategies like SCRUM.
- The game should be fun to play.

1.4. Definitions, acronyms, and abbreviations

Definition	Explanation
Godot	Game engine used to create video games
NPC	Non-Player-Character: Entity controlled by the system
HP	Hit-Points
SFX	Sound effects
HUD	Heads Up Display: UI overlay, which is displayed in-game

Definition	Explanation
FPS	Frames per Second: Unit of Framerate

1.5. References

- GitHub: https://github.com/nilssck/IFP-Tanks/
- Miro: https://miro.com/app/board/uXjVNcPgUHU=/?share_link_id=453941137888
- Trello: https://trello.com/b/wXEjhkif/orga
- Jira: https://ifp-tanks.atlassian.net/jira/software/projects/IT/boards/1/
- GitMind: https://gitmind.com/app/planets/n3a0lcg

1.6. Overview

The project "IFP Tanks" is created for the advanced pratical "Agile development of computer games" and aims to deliver an top-down game where the player controls a tank and needs to eliminate all enemy tanks. This document provides initial outlines for scope, success criteria and requierements.

2. Proposed system

2.1. Overview

2.2. Functional requirements

2.2.1. Player

The player controls a tank in a top-down view which can be upgraded and equip different guns.

2.2.1.1. Movement and controls

The player has the ability to move his tank in all four directions using the `wasd`-Keys (`w`: move up, `s`: move down, `a`: move left, `d`: move right). He can aim the weapon of his tank using the `mouse`, which position is displayed by a recticle, and shoot the weapon with left-click at the aimed position.

2.2.1.2. Stats

The tank controlled by the player has different stats which affect gameplay:

- **HP**: Hitpoints describe the health of a player. Once they reach 0 the player dies and is reset back to the last checkpoints.
- **MaxHP**: The players current HP can not exceed the maxium HP.
- **Damage**: The damgage that the players weapon deals to enemies.
- **Rate of Fire**: The rate at which the player can shoot his weapon. It is measured in shots per second (sps)

2.2.1.3. Upgrades

The player can use coins, which are dropped by enemies when they get destroyed to buy upgrades in the shop after each level. Following upgrades are available:

- **Healing**: Allows to regenerate a certain ammount of HP. HP can not exceed MaxHP
- **Upgrade MaxHP**: Increases the maximum HP
- **Increase Damage**: Increases the damage dealt by the player

2.2.1.4. Weapons

There are a number of different weapons which are acquirable through the shop using coins aswell. These weapons differ in their shooting mechanics and damage statistics. Following weapons are available:

- **50cal**: This gun shoots a straight flying shell which gets destroyed by hitting a wall.
- **Rocket launcher**: The rocket launcher shoots a rocket which homes to a nearby enemy tank.
- **Laser**: The laser fires instantaneously in a straight line damaging all enemies in it's path. The "projectile" of the laser has no travelling time.
- **Grenade launcher**: The grenade launcher fires in an arc allowing the grenades to travel over a wall and damage all enemies in a small damage radius.

2.2.1.5. Damage and death

The player takes damage when hit by an enemy projectile. It is calculated by following formula: `NewHP = OldHP - Damage`. Once the HP of the player reaches 0 he will die and the game is reset to the last checkpoint.

2.2.2. Enemies

There are different enemies each having different stats, abilities and behaviours. They differ by their color so they can be easily differentiated.

2.2.2.1. Types

- **50cal (Blue)**: This tank shoots a straight flying shell at the player which gets destroyed by hitting a wall.
- **Bouncing (Red)**: This tank shoots a straight flying shell which bounces away when hitting a wall and gets destroyed when hitting a wall the second time.
- **Laser (Yellow)**: This tank shoots a laser after a brief charging period which behaves the same like the players laser.
- **Kamikaze (Black)**: This tank drives straight to the player exploding on impact.
- **Rocket (Green)**: This tank fires a rocket homing at the player. The rocket gets destroyed by hitting a wall.
- **Invisible (White)**: This tank is invisible by default and only appears for a brief period after shootin a shell like the 50cal at the player.

2.2.2.2. Stats

Like the player, each tank has it's own of the following stats:

- **HP**: Hitpoints describe the health of a player. Once they reach 0 the player dies and is reset back to the last checkpoints.

- **MaxHP**: The players current HP can not exceed the maxium HP.
- **Damage**: The damgage that the players weapon deals to enemies.
- **Rate of Fire**: The rate at which the player can shoot his weapon. It is measured in shots per second (sps)

Furthermore the enemies has following stats:

- **Speed**: The speed at which the enemie moves araound the battlefield.

2.2.3 Levels and progression

The game is structured into levels which are grouped into worlds. Each world introduces new enemies and tiles mixing up the gameplay.

2.2.3.1 Level

A level is a distinct, individual section of the game. Following points characterize a level:

- **Environment**: An arrangement of specific tiles on the grid. Each level is created and designed by hand.
- **Enemies**: Enemies can be placed freely on the grid. They can not be placed on wall-tiles.
- **Objectives**: The objective of the player is to kill all enemies in the level. Once all enemies are defeated, the level is completed and the shop is opened.

2.2.3.1.1 Grid design

Each level is divided into a grid. On each cell a tile must be placed. Each level is hand-crafted by placing all tiles manually.

2.2.3.1.2 Tiles

Following tiles are available and introduced step by step in each world:

- **Ground**: The ground tile allows the player to freely maneuver over it.
- **Wall**: A wall stops all player movement and generally stops all projectiles (see weapons/enemies for more).
- **Hole**: A hole stops all player movement but dont stop projectiles at all.
- **Switches & Gates**: At first a gate behaves like a wall. When the player drives over a switch the gate opens and starts behaving like a ground tile. This can be reversed by driving over the switch again.
- **Destroyable Walls**: This wall has a certain number of HP. They can be damaged by all projectiles. Once their HP reaches 0 they are destroyed and behave like ground tiles.

2.2.3.2 World

Each world contains a certain number of levels. They differentiate between each other by intruducing new tiles and enemies.

2.2.3.2.1 Checkpoints

Certain levels can be set to be a checkpoint. Once completing a checkpoint level, the game is saved. When the player dies, he can continue the game from the last checkpoint.

2.2.4 UI

2.2.4.1 Menu and navigation

The menu allows the player to navigate the game. Following screens are available:

- **Main menu**: This screen has following buttons:
  - `Start/Continue`: When no save-file is detected, the button shows the text `Start`. When clicking it, the player starts a new game and the first level is loaded. When a save-file is detected, the button shows `continue`, allowing the player to continue the game the last checkpoint saved in the save-file. -`Options`: Clicking this button opens the settings menu.
  - `Exit`: Clicking this button exits the game.
- **Settings**: The settings menu can be reached by the main screen. It allows the player to change following settings:
  - `Music volume`: A slider to change the volume of the music
  - `SFV volume`: A slider to change the volume of the SFX
  - `Back`: A button to return to the main menu
- **Shop**: The shop can be used by the player to buy different upgrades and new weapons (see section Upgrades and Weapons)after a level is completed.
  - A list is of all upgrades and weapons with their corresponding prices is shown. The play can click the upgrade/wapon to buy and equip it.
  - `Continue`: Clicking this button takes the player to the next level.
- **Pause menu**: The pause menu can be reached in-game by pressing the pause-button. It has following buttons:
  - `Continue`: Continue the game. Pressing the pause button again has the same effect.
  - `Exit`: Return to main menu. The game is not saved.

2.2.4.2 HUD

While in-game, following information is displayed to the player on the HUD:

- Current HP of the player displayed in a healthbar.
- Number of remaining enemies
- Number of coins
- Current level

2.3. Nonfunctional requirements

2.3.1. User interface and human factors

The user interface and gameplay should be self-explanatory and easy to learn. Therefore following points shold be followed:

- Buttons should be placed in plausible positions.
- New gameplay elements are intruduced by a self-explanatory level. Text is only used as a last resort.
- A tutorial is not needed, only the controls are shown in the first level.

2.3.2. Documentation

Each step in development should be documented. To achieve this, following points must be considered and implemented:

- Version control
- Task management
- Documents
    - Requirement Analysis
    - Diagrams
    - Report
- Clean, documented and readable code

2.3.3. Hardware and performance characteristics

The game is accessible for all windows and linux enviroments and runs at smooth 60 FPS on low end (to be further defined) hardware.

2.3.4. Error handling and extreme conditions

Error handling is important, the game must not crash on any circumstances. If an critical error occurs the player is notified and the system resets to an acceptable state.

2.3.5. Quality issues

To ensure code quality, following criteria must be considered:

- **Test Driven Development**: Each critical line of code must be tested by at least one unit test.
- **Documented Code**: Code, which is not easily understandable is commented.
- **Code Review**: In order to perform a pull request into the development branch, at least one other person must review the code and approve the changes.

2.3.6. System modifications

Modding support is not officially supported.

2.5. System models

see Folder "documentation"

2.5.5. User-interface -- navigational paths and screen mock-ups

see folder "documentation"

3. Glossary